Автогонки

Индивидуальное домашнее задание 3

Архитектура компьютеров и операционные системы

Вариант 20

Версия 3

Выполнил Куслин Глеб Витальевич

Группа БПИ252

2026

1 Условие и состав решения

Программа моделирует гонку на кольцевой клеточной трассе. Каждый водитель выбирает допустимый маршрут, а судья определяет очередь и бюджет ходов, ведёт гонку и формирует результаты. Работа выполнена на C для Linux. В третьей версии взаимодействие участников организовано через процессы.

По условию команды стартуют на внешней дорожке в порядке жеребьёвки. В начале каждого раунда первой ходит лидирующая машина; при равном продвижении преимущество у внутренней дорожки. Первая машина получает бюджет 4. Бюджет второй равен фактической стоимости хода первой плюс 2, остальных — стоимости предыдущего хода плюс 1.

Разрешены движение вперёд и боковые переходы по свободным клеткам. Вперёд и внутрь стоят по 1, наружу — 3 единицы. Запрещены движение назад, диагонали, занятые клетки и повтор клетки в одном ходе. Машина без допустимого перехода выполняет нулевой ход и сходит после конца раунда. Гонка заканчивается после финиша или схода всех машин либо по пределу раундов. Формируются личная и командная классификации.

Требования ИДЗ 3 предусматривают несколько накопительных реализаций. Все они включены в проект и используют общие правила гонки.

Программы	**Создание участников**	**Обмен и синхронизация**
race	Судья вызывает fork	POSIX-память и неименованные семафоры
race-named	Судья вызывает fork	POSIX-память и именованные семафоры
race-judge и race-car	Независимый запуск	POSIX-память и именованные семафоры
race-mq-judge и race-mq-car	Независимый запуск	POSIX-очереди сообщений и именованные семафоры

Для T команд по C машин работают N = T × C процессов машин и один судья. По умолчанию это 7 процессов, при максимальной конфигурации — 33. Команда и трасса представлены данными: собственного активного поведения у них нет. Судья выдаёт задания, машина выбирает маршрут и выполняет переходы. В независимых вариантах машины не являются детьми судьи.

Ходы выполняются последовательно, поскольку бюджет следующего зависит от предыдущего. Отдельные процессы реализуют самостоятельных участников и обмениваются событиями; одновременное движение нескольких машин правилами этой модели не предусмотрено.

2 Модель трассы и правила

Трасса хранится как массив track[дорожка][клетка]. Значение 0 обозначает свободную клетку, другое значение — ID машины. Дорожки имеют одинаковую длину; боковой переход сохраняет сектор. Первая дорожка внутренняя, последняя внешняя. После последней клетки следует первая. Это упрощение геометрии позволяет явно проверять соседство и занятость.

Car хранит ID, команду, дорожку, клетку, общее продвижение, состояние, признак нулевого хода и место финиша. Race объединяет конфигурацию, машины, занятость, очередь и счётчики. Route содержит направления, число переходов и их стоимость. Локальные ресурсы процессов отделены от модели.

Перед стартом команды перемешиваются по seed. Сначала размещаются первые машины команд, затем вторые и последующие. Машины стоят перед линией старта на разных клетках внешней дорожки. Их продвижение равно -1, -2 и далее; пересечение стартовой линии даёт 0. Финиш наступает при продвижении, равном length × laps. Продвижение увеличивается только при движении вперёд. Боковой переход меняет дорожку и расходует бюджет, но не приближает финиш.

Greedy выбирает вперёд, затем внутрь, затем наружу. Random случайно выбирает из допустимых направлений. Каждая машина инициализирует свой генератор значением seed + id. Поэтому порядок подключения процессов не меняет гонку в одном окружении. Последовательность random отличается от версии 2, где использовался общий генератор.

Инвариант	**Как обеспечивается**
В клетке не более одной машины	Маршрут исключает занятые клетки, переход обновляет старую и новую клетку
Нет движения назад и диагоналей	Доступны только вперёд, внутрь и наружу
Стоимость не превышает бюджет	Каждый переход проверяется по оставшемуся бюджету
Очередь меняется после раунда	Судья сохраняет состав и порядок до конца раунда
Выбывшие больше не ходят	Разрешения получают только машины ACTIVE

Исходная клетка отмечена посещённой; вернуться в неё в этом ходе нельзя. Невозможность продолжить уже непустой маршрут не означает нулевой ход. Финишировавшие и машины с нулевым ходом остаются препятствиями до конца раунда. Затем судья удаляет их с трассы и обновляет очередь.

Финишировавшие ранжируются по порядку финиша. Остальные — по продвижению, затем внутренней дорожке и ID. Место команды определяется лучшей машиной. При остановке по лимиту активные машины получают состояние «не завершила». Круговые премии и дополнительное снятие отставших отключены; это указано в начале вывода программы.

3 Реализации с общей памятью

Первые три реализации используют shm_open, ftruncate и mmap с MAP_SHARED. В SharedRace находятся Race, данные события и кадра, бюджет и стоимость хода, атомарные поля готовности, регистрации и остановки. Локальная структура RaceProcesses хранит PID, pidfd, открытые семафоры и учёт ресурсов.

Модуль	**Назначение**
main.c	Параметры судьи, запуск, управление гонкой и результаты
car_main.c	Вход независимо запускаемой машины
race.c и race.h	Расстановка, маршруты, переходы, раунды и классификация
processes.c и processes.h	Общая память, семафоры, регистрация и завершение
queues.c и queues.h	Передача данных через POSIX-очереди
display.c и display.h	Текст, журнал, поклеточные кадры и терминал

Для N машин нужны N + 2 семафора: permission[i], ready и consumed. Все начальные значения равны 0. В race семафоры размещены в общей памяти и инициализируются sem_init с pshared=1. В race-named и независимой паре используется sem_open для именованных POSIX-семафоров.

1. Судья фиксирует очередь раунда, записывает бюджет и публикует permission.
2. Машина выбирает маршрут и передаёт текст и кадры как TEXT и FRAME.
3. После ready судья читает событие и выводит его. Машина ждёт consumed, поэтому модель остаётся стабильной во время отображения кадра.
4. В конце хода машина передаёт DONE и фактическую стоимость. Затем ждёт следующее разрешение; отдельного подтверждения DONE нет.
5. Судья вычисляет следующий бюджет. После всей очереди выполняет отложенное снятие и пересчитывает порядок.

Право изменять модель передаётся через семафоры. В каждый момент есть не более одного игрового задания, поэтому дополнительный mutex не нужен. Ожидающие процессы блокируются, а не опрашивают общую память в цикле.

В кадре хранится индекс машины, а не адрес Car. Независимые процессы могут отобразить память по разным адресам и восстанавливают указатель в своём отображении. Поле log_path в общей конфигурации обнулено; файлом журнала и экраном управляет только судья.

В вариантах с fork дети наследуют отображение и открытые объекты. Имена памяти и именованных семафоров удаляются после открытия, ещё до гонки. Объекты продолжают существовать до закрытия последнего участника. Это также уменьшает число имён, остающихся при аварийном завершении.

4 Независимые процессы и очереди

При независимом запуске судья ожидает N отдельных программ машин. Для общей памяти имя сессии задаёт объект /akos-session-NAME. Судья создаёт его с O_EXCL и правами 0600, инициализирует данные и публикует состояние JOINING. Машина ожидает готовность, открывает семафоры и атомарно занимает слот своего ID. Повторный ID отклоняется. После подключения всех участников начинается гонка.

Имя памяти сохраняется до окончания сессии, чтобы другой судья не занял то же имя. Имена семафоров содержат поколение и удаляются после подключения всех машин. При остановке судья атомарно закрывает регистрационные слоты, поэтому позднее подключение не создаёт участника во время очистки.

В четвёртой реализации разделяемая память не используется. Судья хранит локальную Race, а машина получает её копию в сообщении TURN. После FRAME и DONE машина возвращает обновлённую модель. Адрес файла журнала в передаваемых данных обнулён. Правила выполняются теми же функциями race.c.

Канал	**Назначение**
/akos-mq-NAME-events	Общая очередь HELLO, TEXT, FRAME и DONE, до 4 сообщений
/akos-client-PID	Личная очередь ACCEPT, REJECT, TURN и STOP, одно сообщение
permission	Именованный семафор разрешения машине прочитать задание
ready и ack	Публикация события и подтверждение TEXT или FRAME
cleanup	Выбор одной машины для очистки после гибели судьи

Машина создаёт личные объекты и отправляет HELLO с ID и PID. Судья проверяет ID и состояние процесса, открывает его очередь и семафор, после чего отправляет ACCEPT или REJECT. Когда оба участника открыли личные объекты, их имена удаляются. Общие объекты судьи сохраняются до конца сессии. Размер сообщения ограничен при компиляции 8192 байт.

Сначала отправляется сообщение, затем публикуется семафор. TEXT и FRAME требуют подтверждения чтения, чтобы следующая запись не обгоняла вывод. DONE завершает ход. Семафоры задают порядок, а модель и события передаются через очереди. Тест с принудительным отказом shm_open подтверждает, что четвёртая реализация не зависит от разделяемой памяти.

Обе независимые реализации используют Linux pidfd для наблюдения за судьёй и машинами. Такой дескриптор привязан к конкретному процессу, поэтому повторное использование номера PID не перенаправляет сигнал. Судья ожидает выход через poll, а коды отдельных машин получает их настоящий родитель. Ожидание подключения ограничено параметром --wait.

5 Интерфейс и запуск

Сборка всех шести программ выполняется командой make. Нужны Linux с поддержкой pidfd, GCC и Make. Проверенная среда: Linux 7.2.15 x86_64, GCC 16.2.1, Python 3.14.10. Для очередей нужны доступные POSIX MQ; используются очереди от 1 до 4 сообщений размером не более 8192 байт.

Первые два варианта сами запускают машины:

```
make
./race --delay 120 --log race.log
./race-named --no-animation --strategy random --seed 7
```

Для независимого запуска каждая строка выполняется в своём терминале. В примере участвуют одна команда и две машины:

```
./race-judge --session demo --teams 1 --cars 2 --wait 60
./race-car demo 1
./race-car demo 2
```

Для обмена через очереди используются другие исполняемые файлы:

```
./race-mq-judge --session demo --teams 1 --cars 2 --wait 60
./race-mq-car demo 1
./race-mq-car demo 2
```

Параметр	**Значения и значение по умолчанию**
--teams и --cars	1–8 команд, 1–4 машины; по умолчанию 3 и 2
--lanes и --length	1–6 дорожек, 3–120 клеток; по умолчанию 4 и 24
--laps и --rounds	1–100 кругов, 1–10000 раундов; по умолчанию 2 и 100
--seed и --strategy	0–4294967295 и greedy/random; по умолчанию 1 и greedy
--delay	0–5000 мс; по умолчанию 120
--no-animation и --quiet	Текст без пауз; только параметры и итоги
--log	Путь к полному текстовому журналу
--session и --wait	Имя и ожидание независимого судьи; 1–60 с, по умолчанию 10

На дорожке должно быть больше клеток, чем всего машин. Имя сессии содержит 1–32 латинские буквы, цифры, знаки - и _. ID — от 1 до teams × cars. Машину можно запустить раньше судьи: она ждёт до 10 секунд; третий аргумент меняет это время, например ./race-car demo 1 60. Разным гонкам одного варианта нужны разные имена. Справка доступна через --help.

6 Отображение и завершение

В интерактивном терминале новый кадр показывается после каждого перехода машины в соседнюю клетку. На экране видны клетки, текущая машина, очередь, маршрут, остаток бюджета и событие. Команды различаются цветом, текущая машина дополнительно выделена. Длинная трасса отображается участком вокруг текущей машины. Проверены окна 80×24 и 100×30.

Экраном и журналом управляет судья. В вариантах с памятью машина ждёт подтверждения, пока судья показывает стабильную модель. В варианте с очередями кадр несёт копию состояния. Подробный текст отражает старт, бюджет, маршрут, каждый переход, блокирование, нулевой ход, круг, сход и финиш. Журнал не содержит управляющих последовательностей терминала.

--no-animation имеет приоритет над --delay при любом порядке аргументов. При --delay 0, --quiet, перенаправлении stdout или TERM=dumb нет кадров и пауз. При --quiet журнал всё равно сохраняет подробные события. Ошибка записи, в том числе обнаруженная при закрытии файла, даёт код 1.

Причина завершения	**Действие**
Все машины финишировали или сошли	Итоговая классификация и освобождение ресурсов
Достигнут предел раундов	Активные машины помечаются как не завершившие
SIGINT или SIGTERM	Завершение начатого раунда без начала нового
Ошибка запуска или обмена	Прекращение работы участников, откат созданных ресурсов
Гибель или остановка машины	Обнаружение через pidfd/waitpid либо таймаут обмена

Обработчик сигнала только сохраняет запрос. Печать, ожидание и очистка выполняются обычным кодом. В вариантах с fork сигнал группе принимает судья; дети блокируют SIGINT/SIGTERM. Независимая машина передаёт запрос судье через атомарное поле в памяти либо pidfd_send_signal для очередей. Пауза проверяет запрос каждые 20 мс и отменяется без ожидания всех 5 секунд.

Ожидание семафора проверяет состояние участников с интервалом 100 мс. Три секунды без игрового события считаются ошибкой; время отображения кадра в этот срок не входит. Судья будит участников для выхода, ожидает около секунды и принудительно завершает зависших. Дети собираются waitpid; для независимых участников ожидается pidfd. Затем закрываются объекты.

Коды судьи: 0 — штатный финиш или лимит, 1 — ошибка, 130 — SIGINT, 143 — SIGTERM. После ошибки незаконченный ход не выдаётся за результат. При обработанном завершении курсор и обычный экран восстанавливаются. SIGKILL не обрабатывается; если все независимые участники погибают одновременно, оставшиеся имена сессии некому удалить.

7 Результаты работы

При одинаковых параметрах четыре реализации дают одинаковые результаты. Для восьми конфигураций сравнены полные журналы, для двух максимальных — краткие итоги. Ниже приведены результаты запусков в проверенной среде.

Стандартная гонка

```
./race --quiet
```

Три команды по две машины, четыре дорожки по 24 клетки, два круга, greedy, seed 1. Все машины финишировали за 11 раундов.

Место	Машина	Команда	Продвижение	Состояние
1	2	1	48	Финиш
2	4	2	48	Финиш
3	5	3	48	Финиш
4	6	3	48	Финиш
5	1	1	48	Финиш
6	3	2	48	Финиш

Места команд: 1, 2, 3. Для race-named и обеих независимых пар с теми же параметрами совпали журнал, число раундов и классификация.

Случайная стратегия

```
./race --strategy random --seed 7 --quiet
```

Гонка завершилась за 22 раунда. Порядок финиша: 3, 4, 6, 1, 5, 2. У всех продвижение 48. Команды заняли места в порядке 2, 3, 1. Одинаковый seed воспроизводит результат во всех четырёх вариантах в одном окружении.

Ограничение числа раундов

```
./race --rounds 1 --quiet
```

После одного раунда гонка остановилась по лимиту. Порядок машин: 1, 5, 3, 6, 2, 4; продвижение соответственно 3, 3, 2, 2, 2, 1. Все шесть имеют состояние «не завершила», а не «сход».

В независимых реализациях отдельно проверены подключение машины раньше судьи, повторный ID, неполный состав и две одновременно работающие сессии. Отказ подключения завершается сообщением об ошибке, штатная сессия с другим именем продолжает работать.

8 Проверка реализации

Полная пересборка и проверка выполняются командой make -B test. Сборка использует C11, -Wall -Wextra -Wpedantic -Werror -O2, -pthread и -lrt. Проверка от 8 ноября 2026 года завершилась с кодом 0.

Набор	Проверенные случаи
Модель C	3248 малых маршрутов, инварианты раунда и 200 гонок
CLI	28 восстановленных журналов, две максимальные конфигурации, неверные параметры и ошибки записи
Процессы	70 отказов запуска, 34 отказа destroy, EINTR, 33 отдельных однопоточных процесса
Анимация	4201 кадр и 3370 переходов в четырёх гонках; состояние восстановлено по журналам
Именованный вариант	138 отказов создания процессов, памяти, открытия и освобождения семафоров
Независимая память	34 отказа открытия семафоров, регистрация и изоляция сессий
Очереди	7 отказов открытия очередей/семафоров и успешная гонка при отказе shm_open
Сравнение	10 конфигураций обеих стратегий с разными seed и максимальными параметрами
Завершение	Реальные SIGINT/SIGTERM, SIGKILL/SIGSTOP машины, потеря судьи, таймауты

Для race-named отдельно пройдены те же CLI- и покадровые проверки. Для каждой независимой реализации проверены кадры одной и шести машин, сигнал машине во время пятисекундной паузы судьи, полный журнал при --quiet и ошибка /dev/full. После проверяемых завершений контролируются выход участников и отсутствие имён памяти и семафоров в /dev/shm. Для очередей mq_open должен вернуть ENOENT; монтирование /dev/mqueue не требуется.

Инварианты проверяются не только по результату гонки. Тесты заново считают координаты, стоимость, бюджет и занятость из журнала, затем сравнивают их с кадрами. Проверяются отсроченное снятие, пересчёт очереди, финиш на линии и отсутствие следующих ходов у выбывшей машины.

Для искусственных отказов tests/faults.c собирается в библиотеку и подключается через LD_PRELOAD. Выбранный вызов возвращает ошибку, остальные выполняются настоящей библиотекой. Это позволяет проверить откат после частичного запуска. Искусственный EINTR проверяет повтор ожидания и сам по себе не доставляет сигнал; реальные сигналы проверяются отдельными сценариями.

Все шесть модулей дополнительно проверены GCC -fanalyzer без диагностик. Совпадение реализаций подтверждает сохранение поведения при смене IPC, а независимые тесты модели проверяют правила. Перечисленные проверки охватывают конкретные сценарии и не гарантируют отсутствия всех ошибок. Санитайзеры в итоговый набор не входят.

9 Критическая оценка и исправления

ИИ использован при разборе требований, проектировании взаимодействия, подготовке кода, тестов и документации. Технические решения проверяются по условию и фактическим результатам. Ниже приведены конкретные проблемы, изменения и способы проверки.

Адрес машины в кадре

Проблема: fork сохраняет адреса отображения, а независимые программы могут открыть общую память по разным адресам. Передавать локальный указатель Car между такими процессами нельзя.

Изменение: Frame хранит индекс машины; судья получает адрес в своём отображении. log_path в передаваемой конфигурации обнулён, локальные дескрипторы не входят в общие данные.

Проверка: в независимой версии сверены реальные кадры одной и шести машин с состоянием, восстановленным по журналу.

Поздняя регистрация

Проблема: между проверкой JOINING и занятием слота судья может начать очистку. Поздний участник тогда не попадёт в прежний список завершения.

Изменение: при остановке регистрационные слоты атомарно закрываются, а прежние PID собираются для ожидания. Новый compare_exchange уже не занимает закрытый слот.

Проверка: сценарии неполного состава, таймаута и сигналов при подключении контролируют выход участников и отсутствие оставшихся ресурсов.

Порядок параметров анимации

Проблема из замечаний к предыдущей версии: --no-animation обнулял задержку, а следующий --delay снова включал анимацию.

Изменение: no_animation хранится отдельным флагом и имеет приоритет. Проверка: оба порядка параметров с задержками 1 и 5000 мс проходят терминальные тесты без кадров и пауз. Исправление сохранено в версии 3.

Принятые ограничения

Дорожки имеют одинаковую длину; стратегии выбирают локальные маршруты, не рассчитывая оптимальную гонку целиком. Замена процесса после старта не предусмотрена. Независимые варианты требуют Linux с pidfd; очереди также зависят от доступной квоты POSIX MQ. При гибели только судьи машины очищают ресурсы, но одновременный SIGKILL всех участников может оставить имена: уцелевшего процесса для очистки нет.